

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ**

**Отчет по лабораторной работе
по курсу «Фронтэнд-разработка»**

Выполнила:

Кошурина Виктория Евгеньевна

Санкт-Петербург
2025 г.

Оглавление

1. Введение.....	3
2. Структура проекта и описание файлов	3
2.1. HTML-файлы.....	3
index.html – главная страница.....	4
authors.html – список кулинаров.....	4
blog.html – страница блогера	4
recipe.html – детальная страница рецепта	4
login.html и register.html – формы авторизации и регистрации.....	4
profile.html – личный кабинет пользователя	5
search.html – страница поиска с фильтрацией.....	5
2.2. CSS-файлы	5
style.css	5
responsive.css.....	6
2.3. JavaScript-файлы.....	6
recipes-data.js.....	6
auth.js	6
main.js	6
profile.js	7
recipe.js	7
search.js.....	8
3. Взаимодействие компонентов	8
4. Заключение.....	9

1. Введение

Целью лабораторной работы являлась разработка клиентского веб-приложения — кулинарного портала, с использованием HTML, CSS, JavaScript и локального хранилища браузера (localStorage). Приложение позволяет просматривать рецепты, авторов, регистрироваться, входить в систему, ставить лайки, оставлять комментарии, подписываться на авторов и искать рецепты по фильтрам.

В проекте реализованы следующие страницы:

- Главная (index.html) — отображает популярные рецепты и лучших кулинаров.
- Поиск (search.html) — поиск рецептов по названию, типу, сложности и ингредиентам.
- Кулинары (authors.html) — список всех авторов.
- Блог автора (blog.html) — страница конкретного автора с его рецептами.
- Рецепт (recipe.html) — страница рецепта с ингредиентами, шагами, видео, лайками и комментариями.
- Вход (login.html) — форма входа.
- Регистрация (register.html) — форма регистрации.
- Личный кабинет (profile.html) — профиль пользователя с сохранёнными рецептами, публикациями и подписками.

Все данные (пользователи, авторы, рецепты, лайки, комментарии) хранятся в localStorage, что обеспечивает эмуляцию серверной части.

2. Структура проекта и описание файлов

2.1. HTML-файлы

Каждая HTML-страница имеет общую структуру: `<head>` с метаданными, подключение Bootstrap, иконок, собственных стилей; `<body>` содержит навигационную панель (navbar), основной контент, футер, и подключает скрипты (Bootstrap, общие модули и специфичные для страницы). Навигационная панель едина для всех страниц (копируется), а кнопки входа/профиля динамически обновляются скриптом auth.js через элемент #authButtons.

index.html – главная страница

- Содержит две секции: «Популярные рецепты» (#featuredRecipes) и «Лучшие кулинары» (#topAuthors).
- Элементы заполняются через JavaScript при загрузке страницы (функция loadHomePage в main.js). Отображаются первые три рецепта и три автора из массива данных.

authors.html – список кулинаров

- Контейнер #authorsContainer для вывода всех авторов (из AUTHORS_DATA). Каждый автор представлен карточкой с аватаром, именем, краткой информацией, количеством подписчиков и кнопкой перехода на блог автора.
- Загрузка выполняется функцией loadAuthors из main.js.

blog.html – страница блогера

- Параметр authorId передаётся через URL (например, blog.html?authorId=1).
- Страница отображает информацию об авторе: аватар, имя, описание, количество подписчиков, кнопку подписки. Ниже – блок с рецептами этого автора (#bloggerRecipes).
- Логика: функция loadBlogger в main.js получает автора по ID, обновляет элементы, устанавливает обработчик на кнопку подписки, фильтрует рецепты по authorId.

recipe.html – детальная страница рецепта

- URL содержит id рецепта (например, recipe.html?id=1).
- Отображает: название, описание, изображение, тип, сложность, кнопку лайка счётчиком, список ингредиентов, пошаговые инструкции, встроенное видео (iframe из Rutube или Dzen), блок автора (аватар, имя, подписчики, кнопка подписки), комментарии и форму добавления комментария.
- За обработку отвечает скрипт recipe.js: получение рецепта, изображение, настройка лайков, загрузка комментариев, подписка на автора.

login.html и register.html – формы авторизации и регистрации

- Простые формы с полями. При отправке вызывают функции login или register из auth.js, которые находят пользователя в localStorage, сохраняют текущего пользователя (currentUser) и

перенаправляют в личный кабинет. Без регистрации можно тоже войти, будет показана страница с дефолтным именем и аватаркой.

profile.html – личный кабинет пользователя

- Содержит информацию о пользователе (аватар, имя, email, биографию) с возможностью редактирования (модальное окно).
- Три вкладки (Bootstrap tabs):
 - «Сохранённые» – рецепты, которые пользователь лайкнул (фильтрация по user.likedRecipes).
 - «Мои публикации» – не доработано, планируется добавить в будущем, стоит заглушка.
 - «Мои подписки» – список авторов, на которых подписан пользователь (информация из user.subscriptions и AUTHORS_DATA).
- Редактирование профиля (имя, био) сохраняется через updateUserProfile в auth.js.
- Основная логика в profile.js.

search.html – страница поиска с фильтрацией

- Левая колонка – фильтры: поиск по названию, чекбоксы типа блюда, кнопки сложности, добавление ингредиентов (теги).
- Правая колонка – результаты (#searchResults).
- Фильтрация реализована в search.js: при нажатии «Применить» собираются все выбранные фильтры, и массив рецептов фильтруется по условиям. Ингредиенты добавляются как массив selectedIngredients и проверяются на вхождение в массив recipe.ingredients.

2.2. CSS-файлы

style.css

- Общие стили: сброс отступов, flex-структура для прижатого футера, пастельно-зелёный фон, стили карточек (тень, hover), оформление аватарок (круглые, обводка), комментариев, переопределение цветов Bootstrap-кнопок под зелёную тему.
- Класс .recipe-card обеспечивает одинаковую высоту карточек рецептов с фиксированной высотой изображения и выравниванием кнопки внизу.

responsive.css

- Медиа-запрос для мобильных устройств (max-width: 768px): адаптация навбара, отступы карточек, корректировка табов в профиле.

2.3. JavaScript-файлы

recipes-data.js

- Содержит два глобальных массива: window.RECIPES_DATA (объекты рецептов) и window.AUTHORS_DATA (объекты авторов).
- Каждый рецепт имеет поля: id, title, description, type, difficulty, image, authorId, ingredients (массив строк), steps (массив строк), videoUrl (ссылка на iframe).
- Авторы: id, name, bio, followers, avatar.
- Эти данные используются для инициализации localStorage при первом запуске (если хранилище пустое).

auth.js

- Основной модуль управления пользователями и данными.
- Функции инициализации initStorage() – проверяет наличие ключей в localStorage, при отсутствии записывает начальные данные из RECIPES_DATA и AUTHORS_DATA.
- Работа с пользователями:
 - register, login, logout, getCurrentUser, setCurrentUser, updateUserProfile.
- Работа с рецептами: getAllRecipes, getRecipeById.
- Лайки: likeRecipe, unlikeRecipe, getLikesCount. Хранятся в объекте likes по recipeId.
- Комментарии: getComments, addComment. Хранятся в объекте comments по recipeId.
- Подписки: subscribe, unsubscribe, isUserSubscribedToAuthor. Подписки хранятся в массиве user.subscriptions, а также обновляют счётчик подписчиков у автора (updateAuthorFollowers).
- Все функции работают с localStorage, эмулируя базу данных.

main.js

- Общий скрипт для всех страниц.

- `updateAuthButtons()` – динамически обновляет навбар: если пользователь залогинен, показывает имя и кнопку «Выйти», иначе – ссылки «Вход» и «Регистрация».
- `loadRecipes()`, `createRecipeCard()` – универсальные функции для отрисовки карточек рецептов (используются на разных страницах).
- `loadAuthors()` – отрисовка карточек авторов для `authors.html`.
- `loadBlogger()` – логика страницы блогера.
- `loadHomePage()`, `loadFeaturedRecipes()`, `loadTopAuthors()` – для главной страницы.
- Обработчики форм входа и регистрации (прямо в файле).
- Функция `loadHomePage` загружает первые три рецепта и первых трёх авторов.

profile.js

- Логика личного кабинета.
- Проверка авторизации (редирект на `login`, если пользователь не залогинен).
- `displayProfileInfo` – вывод данных пользователя.
- `loadSavedRecipes` – фильтрация рецептов по `user.likedRecipes`.
- `loadUserPublications` – фильтрация рецептов по `authorId = user.id`.
- `loadUserSubscriptions` – получение авторов, на которых подписан пользователь, из `AUTHORS_DATA` (используя `getAllAuthors`).
- Редактирование профиля через модальное окно – подстановка текущих данных и сохранение через `updateUserProfile`.

recipe.js

- Скрипт для страницы рецепта.
- Получение `recipeId` из URL, загрузка рецепта.
- `displayRecipe` – заполнение всех элементов страницы: название, описание, изображение, ингредиенты, шаги, видео, информация об авторе.
- `setupLikeButton` – настройка лайка: проверяет, лайкнул ли текущий пользователь этот рецепт, при клике вызывает `likeRecipe/unlikeRecipe` и обновляет счётчик.
- `loadComments` – отображение комментариев.

- `setupCommentForm` – добавление комментария через `addComment`.
- Подписка на автора аналогично `blog.html`, но с обновлением счётчика на странице.

search.js

- Фильтрация рецептов.
- Переменная `selectedIngredients` (массив) для хранения добавленных ингредиентов.
- `loadAllRecipes` – изначально показывает все рецепты.
- `applyFilters` – собирает значения фильтров и фильтрует массив рецептов (регистронезависимый поиск по названию, точное совпадение по типу и сложности, проверка наличия всех выбранных ингредиентов).
- `addIngredient`, `removeIngredient`, `updateIngredientsList` – управление тегами ингредиентов.

3. Взаимодействие компонентов

- **Данные:** все данные хранятся в `localStorage`. При первом запуске `auth.js` инициализирует ключи `recipes`, `authors`, `users`, `comments`, `likes`. Дальнейшие операции (добавление пользователя, лайк, комментариев) обновляют соответствующие ключи.
- **Аутентификация:** после входа/регистрации объект пользователя сохраняется в `currentUser`. На всех страницах через `getCurrentUser()` можно получить текущего пользователя. При выходе `currentUser` удаляется.
- **Подписки:** при нажатии «Подписаться» вызывается `subscribe(authorId)`, который добавляет ID автора в массив `user.subscriptions` и увеличивает `author.followers`. При отписке – удаляет и уменьшает счётчик. Обновление счётчика происходит на странице через изменение свойства `author.followers` и запись обратно в `localStorage`.
- **Лайки:** лайк сохраняется в объекте `likes` (просто число) и добавляется/удаляется ID рецепта из `user.likedRecipes`. Это позволяет отображать сохранённые рецепты в профиле.
- **Комментарии:** хранятся в объекте, где ключ – ID рецепта, значение – массив комментариев с полями: `id`, `userId`, `userName`, `text`, `date`.
- **Связь страниц:**

- С главной страницы можно перейти на страницу рецепта (recipe.html?id=...) и блог автора (blog.html?authorId=...).
- Со страницы авторов – на блог автора.
- Со страницы рецепта – на блог автора.
- Из профиля – на рецепты (по клику) и на блоги авторов (из подписок).

4. Заключение

Разработанное приложение представляет собой полноценный клиентский кулинарный портал с широким функционалом: просмотр, поиск, фильтрация, лайки, комментарии, подписки, личный кабинет. Вся логика реализована на чистом JavaScript с использованием localStorage для сохранения данных между сессиями, что делает приложение полностью самодостаточным и не требующим серверной части. Код организован модульно, каждый файл отвечает за свою функциональность. Страницы адаптированы под мобильные устройства благодаря Bootstrap и дополнительным медиа-запросам.

Проект демонстрирует навыки работы с DOM, событиями, формами, модальными окнами, а также умение структурировать код и использовать локальное хранилище для эмуляции базы данных.